

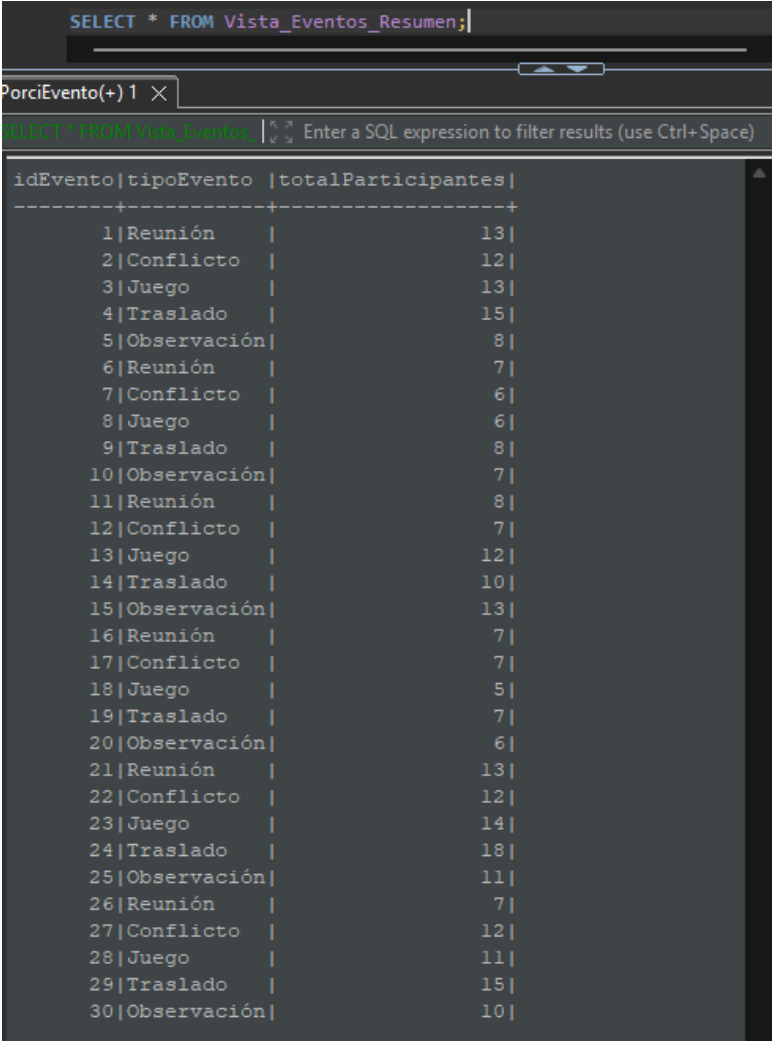
Ejercicio.

Sobre la base de datos PorciTEIS realiza las siguientes Vistas, Funciones, Procedimientos, Triggers asociados:

- **Vistas:**
 - Crea una vista llamada **Vista_Eventos_Resumen** que muestre los eventos junto con el número total de porcinos participantes en cada uno, incluyendo el identificador del evento, tipo y total de participantes.

```
CREATE OR REPLACE VIEW Vista_Eventos_Resumen AS
SELECT
    pe.idEvento,
    pe.tipoEvento,
    COUNT(pec.idPorcino) AS totalParticipantes
FROM PorciEvento pe
JOIN PorciEvento_Cercado pec ON pe.idEvento = pec.idEvento
GROUP BY pe.idEvento, pe.tipoEvento;

SELECT * FROM Vista_Eventos_Resumen;
```

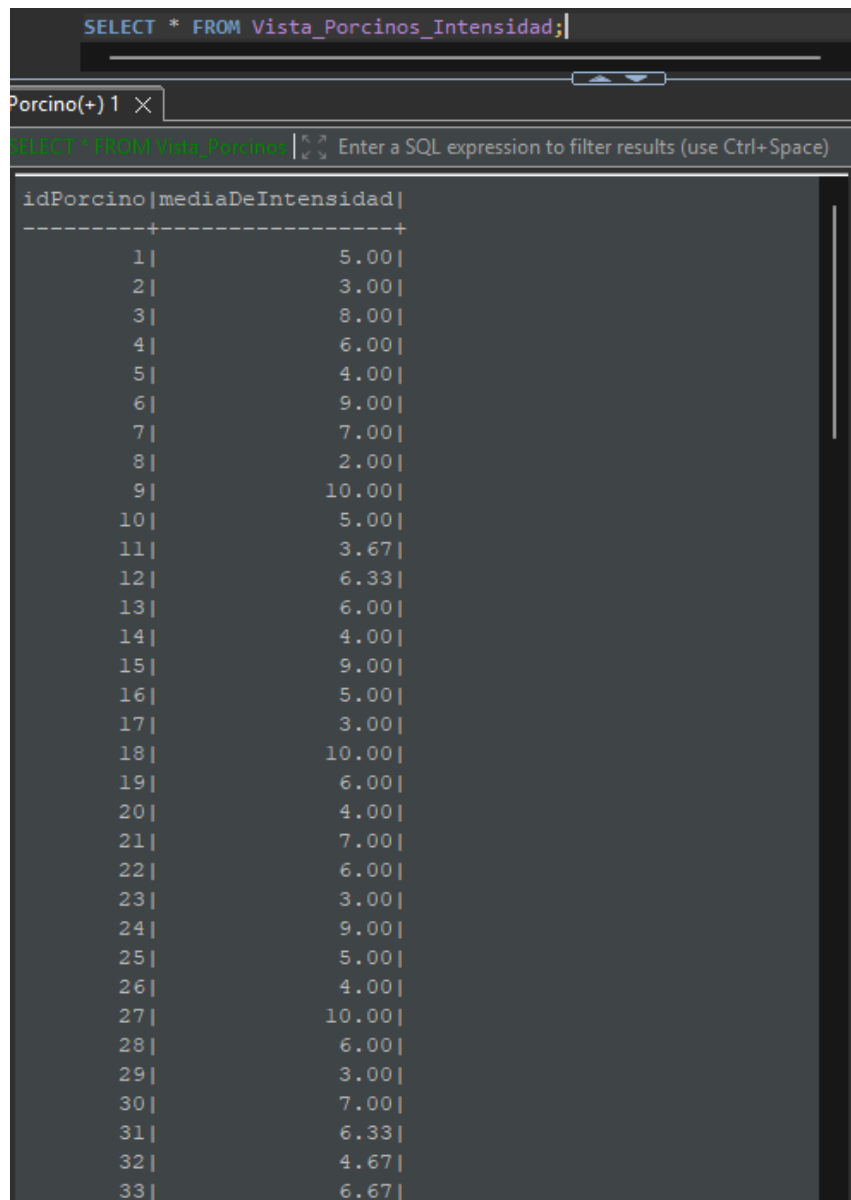


idEvento	tipoEvento	totalParticipantes
1	Reunión	13
2	Conflicto	12
3	Juego	13
4	Traslado	15
5	Observación	8
6	Reunión	7
7	Conflicto	6
8	Juego	6
9	Traslado	8
10	Observación	7
11	Reunión	8
12	Conflicto	7
13	Juego	12
14	Traslado	10
15	Observación	13
16	Reunión	7
17	Conflicto	7
18	Juego	5
19	Traslado	7
20	Observación	6
21	Reunión	13
22	Conflicto	12
23	Juego	14
24	Traslado	18
25	Observación	11
26	Reunión	7
27	Conflicto	12
28	Juego	11
29	Traslado	15
30	Observación	10

- Crea una vista llamada **Vista_Porcinos_Intensidad** que muestre los porcinos junto con la intensidad media de sus participaciones, incluyendo el identificador del porcino y la media de intensidad.

```
CREATE OR REPLACE VIEW Vista_Porcinos_Intensidad AS
SELECT
    p.idPorcino,
    ROUND(AVG(pec.intensidad), 2) AS mediaDeIntensidad
FROM Porcino p
JOIN PorciEvento_Cercado pec ON p.idPorcino = pec.idPorcino
GROUP BY p.idPorcino;

SELECT * FROM Vista_Porcinos_Intensidad;
```



SELECT * FROM Vista_Porcinos_Intensidad;

Porcino(+) 1 X

SELECT * FROM Vista_Porcinos | Enter a SQL expression to filter results (use Ctrl+Space)

idPorcino	mediaDeIntensidad
1	5.00
2	3.00
3	8.00
4	6.00
5	4.00
6	9.00
7	7.00
8	2.00
9	10.00
10	5.00
11	3.67
12	6.33
13	6.00
14	4.00
15	9.00
16	5.00
17	3.00
18	10.00
19	6.00
20	4.00
21	7.00
22	6.00
23	3.00
24	9.00
25	5.00
26	4.00
27	10.00
28	6.00
29	3.00
30	7.00
31	6.33
32	4.67
33	6.67

- **Funciones:**

- Crea una función llamada `obtener_intensidad_media_evento` que reciba el identificador de un evento y devuelva la intensidad media de los porcinos que participaron en dicho evento.

```
#Borro la función por si ya existe:

DROP FUNCTION IF EXISTS obtener_intensidad_media_evento;

#Abro el delimitador para el código de la función

DELIMITER //

CREATE FUNCTION obtener_intensidad_media_evento(p_idEvento INT)
RETURNS DECIMAL(5,2)
READS SQL DATA #Etiqueta recomendada cuando la función hace consultas.
#DETERMINISTIC (Si la función fuese independiente de los datos almacenados)
BEGIN
    #Se declara la variable para guardar el resultado
    DECLARE resultado DECIMAL(5,2);

    #Se hace la consulta y se guarda el valor redondeado en la variable "resultado"
    SELECT ROUND(AVG(intensidad), 2) INTO resultado
    FROM PorciEvento_Cercado pec
    WHERE pec.idEvento = p_idEvento;

    #Se devuelve el resultado
    RETURN resultado;
END

DELIMITER ;

#Visualizamos el resultado para los eventos 1, 5, 13, 21 y 30
SELECT obtener_intensidad_media_evento(1);
SELECT obtener_intensidad_media_evento(5);
SELECT obtener_intensidad_media_evento(13);
SELECT obtener_intensidad_media_evento(21);
SELECT obtener_intensidad_media_evento(30);
```

#Visualizamos el resultado para:

```
obtener_intensidad_media_evento(1) |
-----+
6.15|
```

SELECT obtener_intensidad_media_evento(13);

```
obtener_intensidad_media_evento(13) |
-----+
5.58|
```

SELECT obtener_intensidad_media_evento(5);

```
obtener_intensidad_media_evento(5) |
-----+
6.13|
```

SELECT obtener_intensidad_media_evento(21);

```
obtener_intensidad_media_evento(21) |
-----+
6.00|
```

- Crea una función llamada `obtener_ultima_fecha_porcino` que reciba el identificador de un porcino y devuelva la fecha del último evento en el que ha participado.

```
#Borro la función por si ya existe:

DROP FUNCTION IF EXISTS obtener_ultima_fecha_porcino;

#Abro el delimitador para el código de la función

DELIMITER //

CREATE FUNCTION obtener_ultima_fecha_porcino(p_idPorcino INT)
RETURNS DATE
READS SQL DATA #Etiqueta recomendada cuando la función hace consultas.
#DETERMINISTIC (Si la función fuese independiente de los datos almacenados)
BEGIN
    #Se declara la variable para guardar el resultado
    DECLARE resultado DATE;

    #Se hace la consulta y se guarda la fecha en la variable "resultado"
    SELECT MAX(pec.fechaRegistro) INTO resultado
    FROM PorciEvento_Cercado pec
    WHERE pec.idPorcino = p_idPorcino;

    #Se devuelve el resultado
    RETURN resultado;
END

DELIMITER ;

#Visualizamos el resultado para los porcinos 1, 25, 69, 121 y 199
SELECT obtener_ultima_fecha_porcino(1);
SELECT obtener_ultima_fecha_porcino(25);
SELECT obtener_ultima_fecha_porcino(69);
SELECT obtener_ultima_fecha_porcino(121);
SELECT obtener_ultima_fecha_porcino(199);
```

```
#Visualizamos el resultado por:
obtener_ultima_fecha_porcino(1) |
-----+
2023-01-02 |
```

```
SELECT obtener_ultima_fecha_porcino(69) |
-----+
2023-01-12 |
```

```
SELECT obtener_ultima_fecha_porcino(25) |
-----+
2023-01-05 |
```

```
SELECT obtener_ultima_fecha_porcino(121) |
-----+
2023-01-21 |
```

- **Procedimientos:**

- Crea un procedimiento llamado **mostrar_eventos_por_porcino** que reciba un identificador de porcino y muestre todos los eventos en los que ha participado, indicando tipo y fecha.

```
#Borro el procedimiento por si ya existe:
DROP PROCEDURE IF EXISTS mostrar_eventos_por_porcino;

DELIMITER //

#Creo el procedimiento definiendo el parámetro
CREATE PROCEDURE mostrar_eventos_por_porcino(IN p_idPorcino INT)
BEGIN
    #Consulta con select con un inner join para filtrar los eventos de cada
    idPorcino
    SELECT pe.idEvento, pe.tipoEvento, pe.fechaInicio
    FROM PorciEvento pe
    INNER JOIN PorciEvento_Cercado pec ON pe.idEvento = pec.idEvento
    WHERE pec.idPorcino = p_idPorcino;
END //

DELIMITER ;

#Llamamos al procedimiento con el comando CALL para un porcino concreto.
CALL mostrar_eventos_por_porcino(10);
CALL mostrar_eventos_por_porcino(16);
```

PorciEvento 1 × Statistics 1

#Llamamos al procedimiento :

idEvento	tipoEvento	fechaInicio
2	Conflicto	2023-01-06
22	Conflicto	2023-01-26

PorciEvento 1 × Statistics 1

CALL mostrar_eventos_por_por

idEvento	tipoEvento	fechaInicio
3	Juego	2023-01-07
23	Juego	2023-01-27

- Crea un procedimiento llamado `mostrar_estadisticas_evento` que reciba un tipo de evento y muestre el número total de eventos de ese tipo, junto con la duración media.

```
#Borro el procedimiento por si ya existe:
DROP PROCEDURE IF EXISTS mostrar_estadisticas_evento;

DELIMITER //

#Creo el procedimiento definiendo el parámetro
CREATE PROCEDURE mostrar_estadisticas_evento(IN pe_tipoEvento VARCHAR(30))
BEGIN
    #Consulta con select para contar las coincidencias del tipo de evento y su
    duración media
    SELECT COUNT(pe.idEvento) AS NUMERO_TOTAL_EVENTOS, ROUND(AVG(duracionMinutos)) AS
    DURACION_MEDIA_MINUTOS
    FROM PorciEvento pe
    WHERE pe.tipoEvento = pe_tipoEvento;
END //

DELIMITER ;

#Llamamos al procedimiento con el comando CALL para un tipo de evento concreto.
CALL mostrar_estadisticas_evento("Conflicto");
CALL mostrar_estadisticas_evento("Reunión");
CALL mostrar_estadisticas_evento("Juego");
```

```
#Llamamos al procedimiento : Enter a SQL expression to filter result

NUMERO_TOTAL_EVENTOS|DURACION_MEDIA_MINUTOS|
-----+-----+
6|59|
```

```
CALL mostrar_estadisticas_evento("Reunión"); Enter a SQL expression to filter result

NUMERO_TOTAL_EVENTOS|DURACION_MEDIA_MINUTOS|
-----+-----+
6|51|
```

```
CALL mostrar_estadisticas_evento("Juego"); Enter a SQL expression to filter result

NUMERO_TOTAL_EVENTOS|DURACION_MEDIA_MINUTOS|
-----+-----+
6|40|
```

- **Cursores:**

- Crea un procedimiento llamado `cursor_porcinos_intensidad` que utilice un cursor implícito para recorrer los porcinos y mostrar aquellos cuya intensidad media de participación sea superior a un valor dado como parámetro.

```
# Borro el procedimiento por si ya existe
DROP PROCEDURE IF EXISTS cursor_porcinos_intensidad;

DELIMITER //

CREATE PROCEDURE cursor_porcinos_intensidad(IN p_umbral_intensidad DECIMAL(5,2))
BEGIN
    -- 1. VARIABLES
    -- Necesitamos una variable por cada columna que devuelva nuestro SELECT
    DECLARE v_idPorcino INT;
    DECLARE v_intensidad DECIMAL(5,2);
    DECLARE fin_cursor INT DEFAULT 0; -- Variable para finalizar
    -- 2. CURSOR
    -- Incluye un SELECT
    DECLARE c_porcinos CURSOR FOR
        SELECT pec.idPorcino, ROUND(AVG(pec.intensidad)) AS INTENSIDAD_MEDIA
        FROM PorciEvento_Cercado pec
        GROUP BY pec.idPorcino
        HAVING INTENSIDAD_MEDIA > p_umbral_intensidad;
    -- 3. HANDLER(Manejo de excepciones durante la ejecución del bucle)
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = 1;
    #Creamos una tabla temporal vacía para ir guardando los datos porque MySQL no
    tiene un comando directo para mostrar los datos generados por el bucle
    DROP TEMPORARY TABLE IF EXISTS Temp_Resultados;
    CREATE TEMPORARY TABLE Temp_Resultados (idPorcino INT, intensidad_media
    DECIMAL(5,2));
    -- 4. BUCLE DE LECTURA
    OPEN c_porcinos;
    mi_bucle: LOOP
        -- A. Sacar datos del cursor (FETCH)
        FETCH c_porcinos INTO v_idPorcino, v_intensidad;
        -- B. Comprobar semáforo de salida
        IF fin_cursor = 1 THEN
            LEAVE mi_bucle;
        END IF;
        -- C. Insertar los datos de las variables en la tabla temporal
        INSERT INTO Temp_Resultados (idPorcino, intensidad_media) VALUES
        (v_idPorcino, v_intensidad);
    END LOOP mi_bucle;
    CLOSE c_porcinos;
    -- 5. Mostrar los resultados acumulados
    SELECT * FROM Temp_Resultados;
END //

DELIMITER ;

# Llamada de prueba (por ejemplo, buscar porcinos con intensidad media superior a 6.5)
CALL cursor_porcinos_intensidad(6.5);
```

```
idPorcino|intensidad_media|
-----+-----+
      3|          8.00|
      6|          9.00|
      7|          7.00|
      9|         10.00|
     15|          9.00|
     18|         10.00|
     21|          7.00|
     24|          9.00|
     27|         10.00|
     30|          7.00|
     33|          7.00|
     36|          9.00|
     39|          8.00|
     42|          8.00|
     45|          9.00|
     48|         10.00|
     51|          9.00|
     54|          8.00|
     57|         10.00|
     60|          9.00|
     63|          8.00|
     66|          9.00|
     69|          8.00|
     72|          9.00|
     75|         10.00|
     78|          8.00|
     81|          9.00|
     84|         10.00|
     87|          8.00|
     90|          9.00|
     93|         10.00|
     96|          9.00|
```


- Crea un procedimiento llamado `cursor_eventos_mas_5` que utilice un cursor explícito para recorrer los eventos ordenados por fecha y mostrar únicamente los que tengan más de 5 participantes.

```
# Borro el procedimiento por si ya existe
DROP PROCEDURE IF EXISTS cursor_eventos_mas_5;
DELIMITER //

CREATE PROCEDURE cursor_eventos_mas_5()
BEGIN
    -- 1. VARIABLES
    -- Necesitamos una variable por cada columna que devuelva el SELECT
    DECLARE v_idEvento INT;
    DECLARE v_fechaInicio DATE;
    DECLARE v_participantes INT;
    DECLARE fin_cursor INT DEFAULT 0; -- Variable para finalizar
    -- 2. CURSOR
    -- Incluye una consulta SELECT
    DECLARE c_eventos CURSOR FOR
        SELECT pe.idEvento, pe.fechaInicio, COUNT(pec.idPorcino) AS
NUMERO_PARTICIPANTES
        FROM PorciEvento pe
        INNER JOIN PorciEvento_Cercado pec ON pe.idEvento = pec.idEvento
        GROUP BY pe.idEvento
        HAVING NUMERO_PARTICIPANTES > 5
        ORDER BY pe.fechaInicio;
    -- 3. HANDLER(Manejo de excepciones durante la ejecución del bucle)
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = 1;
    #Creamos una tabla temporal vacía para ir guardando los datos porque MySQL no
    tiene un comando directo para mostrar los datos generados por el bucle:
    DROP TEMPORARY TABLE IF EXISTS Temp_Eventos;
    CREATE TEMPORARY TABLE Temp_Eventos (idEvento INT, fechaInicio DATE, participantes
INT);
    -- 4. BUCLE DE LECTURA
    #Se "ejecuta" el cursor que realiza la consulta SELECT
    OPEN c_eventos;
    mi_bucle: LOOP
        -- A. Sacar datos del cursor (FETCH)
        FETCH c_eventos INTO v_idEvento, v_fechaInicio, v_participantes;
        -- B. Comprobar semáforo de salida
        IF fin_cursor = 1 THEN
            LEAVE mi_bucle;
        END IF;
        -- C. Insertar los datos de las variables en la tabla temporal
        INSERT INTO Temp_Eventos (idEvento, fechaInicio, participantes) VALUES
(v_idEvento, v_fechaInicio, v_participantes);
    END LOOP mi_bucle;
    CLOSE c_eventos;
    -- 5. Mostrar los resultados acumulados
    SELECT * FROM Temp_Eventos;
END //
DELIMITER ;
# Llamada de prueba
CALL cursor_eventos_mas_5();
```

Tabla 1: Eventos y Participantes		
idEvento	fechaInicio	participantes
1	2023-01-05	13
2	2023-01-06	12
3	2023-01-07	13
4	2023-01-08	15
5	2023-01-09	8
6	2023-01-10	7
7	2023-01-11	6
8	2023-01-12	6
9	2023-01-13	8
10	2023-01-14	7
11	2023-01-15	8
12	2023-01-16	7
13	2023-01-17	12
14	2023-01-18	10
15	2023-01-19	13
16	2023-01-20	7
17	2023-01-21	7
19	2023-01-23	7
20	2023-01-24	6
21	2023-01-25	13
22	2023-01-26	12
23	2023-01-27	14
24	2023-01-28	18
25	2023-01-29	11
26	2023-01-30	7
27	2023-01-31	12
28	2023-02-01	11
29	2023-02-02	15
30	2023-02-03	10

- **Triggers:**

- Crea un trigger llamado `trigger_check_evento` que no deje insertar registros en la tabla `PorciEvento_Cercado` si el identificador del evento no existe dentro de la tabla `PorciEvento`.

```
# Borramos el trigger si existe
DROP TRIGGER IF EXISTS trigger_check_evento;

DELIMITER //

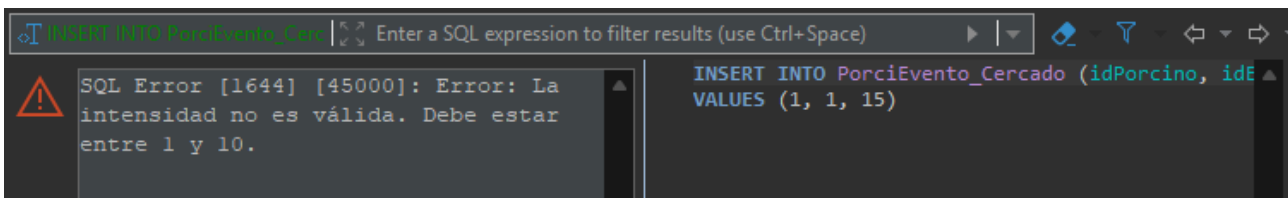
CREATE TRIGGER trigger_check_evento
BEFORE INSERT ON PorciEvento_Cercado
FOR EACH ROW #Se comprueba fila a fila.
BEGIN
    #1. Declaramos una variable para guardar el evento si existe
    DECLARE v_existe INT;

    #2. Hacemos la consulta contando las coincidencias
    SELECT COUNT(*) INTO v_existe
    FROM PorciEvento pe
    WHERE pe.idEvento= NEW.idEvento;

    #3. Si no existe (es 0), lanzamos el error
    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El identificador del evento no existe en la tabla
PorciEvento.';
    END IF;
END //

DELIMITER ;

# Prueba 5.A: Forzar inserción con evento inexistente
INSERT INTO PorciEvento_Cercado (idPorcino, idEvento, intensidad)
VALUES (1, 9999, 5);
-- Resultado esperado: La consulta debe fallar y el gestor de BBDD mostrará:
-- "Error: El identificador del evento no existe en la tabla PorciEvento."
```



- Crea un trigger llamado `trigger_validar_intensidad` que se ejecute antes de insertar o actualizar registros en la tabla `PorciEvento_Cercado`.

El trigger deberá garantizar que el valor del campo `Intensidad` esté comprendido entre 1 y 10 (ambos incluidos). En caso de que el valor introducido sea inferior a 1 o superior a 10, el sistema deberá impedir la operación y lanzar un mensaje de error indicando que la intensidad no es válida.

Este control tiene como objetivo asegurar la integridad de los datos relativos a la participación de los porcinos en los eventos.

```
# 2. TRIGGER PARA CUANDO SE ACTUALIZA UN REGISTRO EXISTENTE

DROP TRIGGER IF EXISTS trigger_validar_intensidad_update;

DELIMITER //
CREATE TRIGGER trigger_validar_intensidad_update
BEFORE UPDATE ON PorciEvento_Cercado
FOR EACH ROW
BEGIN
    IF NEW.intensidad < 1 OR NEW.intensidad > 10 THEN #NEW.intensidad sería un nuevo
valor para sustituir el anterior.
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La intensidad no es válida. Debe estar entre 1 y
10.';
    END IF;
END //
DELIMITER ;

# Prueba (UPDATE): Forzar actualización con intensidad inferior a 1

UPDATE PorciEvento_Cercado
SET intensidad = 0
WHERE idPorcino = 1 AND idEvento = 1;
```

